

** BKAV Invoice Worker*

** Worker chạy trên VPS FPT, poll canister IC mỗi 15 giây để lấy đơn hàng cần phát hành hóa đơn,
* sau đó gọi BKAV API để phát hành và ghi kết quả về canister qua HTTP callback.*

** Sử dụng @dfinity/agent để gọi query call đến IC canister.*

** Endpoint: JSON POST (không dùng SOAP XML wrapper)*

**/*

```
import { HttpAgent, Actor } from '@dfinity/agent';
import { Ed25519KeyIdentity } from '@dfinity/identity';
import { Principal } from '@dfinity/principal';
import { IDL } from '@dfinity/candid';
import https from 'https';
import http from 'http';
import crypto from 'crypto';
import { readFileSync, writeFileSync, existsSync, chmodSync } from 'node:fs';
import path from 'node:path';
// — Configuration
```

```
const CANISTER_ID = process.env.CANISTER_ID || '';
const IC_HOST = process.env.IC_HOST || 'https://icp0.io';
const POLL_INTERVAL_MS = parseInt(process.env.POLL_INTERVAL_MS || '15000');
const LOG_LEVEL = process.env.LOG_LEVEL || 'info';
const INVOICE_CALLBACK_SECRET = process.env.INVOICE_CALLBACK_SECRET || '';
const WORKER_IDENTITY_PATH = process.env.WORKER_IDENTITY_PATH ||
'./worker-identity.json';
if (!CANISTER_ID) {
  console.error('[bkav-worker] ERROR: CANISTER_ID is required. Set it via environment
variable.');
```

process.exit(1);

```
}
if (!INVOICE_CALLBACK_SECRET) {
  console.warn('[bkav-worker] WARN: INVOICE_CALLBACK_SECRET not set - callback auth will
fail');
```

}

```
// — Identity Persistence
```

```
// Worker phải có principal cố định qua các lần restart VPS. Ed25519KeyIdentity
// được sinh một lần (lần chạy đầu), lưu ra file JSON, và nạp lại ở các lần sau.
// File mặc định ./worker-identity.json (override bằng env WORKER_IDENTITY_PATH).
// File chứa cặp khóa — KHÔNG commit vào git, chmod 0600 để giới hạn truy cập.
function loadOrCreateIdentity(identityPath) {
  const resolvedPath = path.resolve(identityPath);
  // Nếu file đã tồn tại → nạp lại identity cũ để giữ principal.
  if (existsSync(resolvedPath)) {
    try {
      const jsonStr = readFileSync(resolvedPath, 'utf8');
      const identity = Ed25519KeyIdentity.fromJSON(jsonStr);
      const principalText = identity.getPrincipal().toText();
      log('info', `Loaded persisted worker identity from ${resolvedPath}`);
```

```

    log('info', `Worker principal: ${principalText}`);
    log('info', 'Register this principal with the canister owner so the worker can be authorized.');
```

return identity;

```

} catch (err) {
  // File hỏng (parse lỗi, nội dung không hợp lệ) → log warning và sinh identity mới.
  log('warn', `Failed to load identity from ${resolvedPath}: ${err.message}. Generating a new
identity.`);
}
} else {
  log('info', `No identity file at ${resolvedPath}. Generating a new Ed25519KeyIdentity.`);
}
// Sinh identity mới và ghi ra file.
const identity = Ed25519KeyIdentity.generate();
const principalText = identity.getPrincipal().toText();
const jsonStr = JSON.stringify(identity.toJSON());
try {
  writeFileSync(resolvedPath, jsonStr, { mode: 0o600 });
  // Đảm bảo quyền 0600 (chỉ owner đọc/ghi) — writeFileSync mode có thể không
  // áp dụng đầy đủ trên mọi filesystem, nên chmodSync lại để chắc chắn.
  try {
    chmodSync(resolvedPath, 0o600);
  } catch (chmodErr) {
    log('warn', `Could not chmod 0600 on ${resolvedPath}: ${chmodErr.message}`);
  }
  log('info', `Saved new worker identity to ${resolvedPath} (chmod 0600)`);
} catch (writeErr) {
  log('error', `Failed to write identity file ${resolvedPath}: ${writeErr.message}`);
  log('warn', 'Worker will continue with an in-memory identity — principal will change on restart.');
```

}

```

log('info', `Worker principal: ${principalText}`);
log('info', 'Register this principal with the canister owner so the worker can be authorized.');
```

return identity;

```

}
// — IDL Factory (inline)
```

```

const idlFactory = ({ IDL }) => {
  const InvoiceItem = IDL.Record({
    name: IDL.Text,
    unit: IDL.Opt(IDL.Text),
    itemNote: IDL.Opt(IDL.Text),
    quantity: IDL.Nat,
    price: IDL.Nat,
    menuItemId: IDL.Nat,
  });
  const VatInfo = IDL.Record({
    taxCode: IDL.Opt(IDL.Text),
    buyerName: IDL.Text,
    address: IDL.Text,
    email: IDL.Text,
    accountNo: IDL.Opt(IDL.Text),
  });

```

```

});
// IDL must match backend PendingInvoiceItem (bkav-invoice-api.mo) exactly.
// Backend returns 15 fields: orderId, restaurantId, items, totalAmount, createdAt,
// vatInfo, buyerName, buyerAddress, customerTaxCode, customerCompanyName,
// customerCompanyAddress, isDemo, isRetailInvoice.
// buyerName/buyerAddress are derived ONLY from vatInfo (empty when retail).
// isRetailInvoice = true → issue retail receipt (Bán cho người tiêu dùng); false → VAT invoice.
const PendingInvoiceItem = IDL.Record({
  orderId: IDL.Nat,
  restaurantId: IDL.Nat,
  items: IDL.Vec(InvoiceItem),
  totalAmount: IDL.Nat,
  createdAt: IDL.Int,
  vatInfo: IDL.Opt(VatInfo),
  buyerName: IDL.Text,
  buyerAddress: IDL.Text,
  customerTaxCode: IDL.Opt(IDL.Text),
  customerCompanyName: IDL.Opt(IDL.Text),
  customerCompanyAddress: IDL.Opt(IDL.Text),
  isDemo: IDL.Bool,
  isRetailInvoice: IDL.Bool,
});
const WorkerConfig = IDL.Record({
  bkavProdEndpoint: IDL.Text,
  bkavDemoEndpoint: IDL.Text,
  partnerGUID: IDL.Text,
  partnerToken: IDL.Text,
  invoiceSerial: IDL.Text,
  demoInvoiceSerial: IDL.Text,
  prodInvoiceSerial: IDL.Text,
  invoiceForm: IDL.Text,
  vatRate: IDL.Float64,
  useDemo: IDL.Bool,
  demoGuid: IDL.Text,
  demoToken: IDL.Text,
  realGuid: IDL.Text,
  realToken: IDL.Text,
  invoiceCallbackSecret: IDL.Text,
});
return IDL.Service({
  getPendingInvoices: IDL.Func([], [IDL.Vec(PendingInvoiceItem)], ['query']),
  getPendingDemoInvoices: IDL.Func([], [IDL.Vec(PendingInvoiceItem)], ['query']),
  getInvoiceWorkerConfig: IDL.Func([], [WorkerConfig], ['query']),
});
};
// — Agent Setup

```

—

*// Nạp (hoặc sinh mới) identity cố định cho worker trước khi tạo HttpAgent,
 // để worker có principal ổn định qua các lần restart VPS.*

```
const identity = loadOrCreateIdentity(WORKER_IDENTITY_PATH);
const agent = new HttpAgent({ host: IC_HOST, identity });
if (IC_HOST.includes('localhost') || IC_HOST.includes('127.0.0.1')) {
  await agent.fetchRootKey();
}
const actor = Actor.createActor(idlFactory, {
  agent,
  canisterId: Principal.fromText(CANISTER_ID),
});
// — Dedup Map
```

```
const dedupMap = new Map(); // orderId -> timestamp
const DEDUP_TTL_MS = 120000;
const MAX_RETRIES = 3;
const RETRY_DELAY_MS = 5000;
function isDuplicate(orderId) {
  const ts = dedupMap.get(orderId);
  if (!ts) return false;
  if (Date.now() - ts > DEDUP_TTL_MS) {
    dedupMap.delete(orderId);
    return false;
  }
  return true;
}
function markProcessing(orderId) {
  dedupMap.set(orderId, Date.now());
}
// — Logging
```

```
function log(level, msg, data) {
  if (level === 'debug' && LOG_LEVEL !== 'debug') return;
  if (level === 'error' && LOG_LEVEL === 'none') return;
  const ts = new Date().toISOString();
  const line = data
    ? `[bkav-worker] ${ts} [${level.toUpperCase()}] ${msg} ${JSON.stringify(data)}`
    : `[bkav-worker] ${ts} [${level.toUpperCase()}] ${msg}`;
  console.log(line);
}
// — JSON Payload Builder
```

```
// Builds the inner JSON object to be Base64-encoded and sent as CommandData.
// All field names use camelCase exactly as BKAV API requires.
// invoiceForm is always "" — BKAV assigns it via CmdType 100.
// invoiceSerial is "" in demo mode (BKAV auto-assigns); in production mode it
// uses config.prodInvoiceSerial when provided, otherwise "" for auto-assign.
//
// Field mapping (matches backend PendingInvoiceItem in bkav-invoice-api.mo):
```

```

// - isRetailInvoice = true → retail receipt (Bán cho người tiêu dùng): buyerName="Bán cho người
tiêu dùng", empty tax/address
// - isRetailInvoice = false → VAT invoice: use buyerName, buyerAddress, customerTaxCode,
// customerCompanyName, customerCompanyAddress from vatInfo
// - buyerName/buyerAddress are derived ONLY from vatInfo by the backend (empty when retail)
function buildJsonPayload(invoice, config) {
  const orderId = invoice.orderId;
  // vatInfo takes priority — when present and taxCode non-empty, it's a B2B VAT invoice
  const vatInfo = invoice.vatInfo.length > 0 ? invoice.vatInfo[0] : null;
  const vatTaxCode = vatInfo && vatInfo.taxCode.length > 0 ? vatInfo.taxCode[0] : null;
  const vatBuyerName = vatInfo ? vatInfo.buyerName : null;
  const vatAddress = vatInfo ? vatInfo.address : null;
  const vatEmail = vatInfo && vatInfo.email && vatInfo.email.length > 0 ? vatInfo.email : null;
  // Unwrap flat optional fields (Candid Opt is [] or [value] in JS)
  const customerTaxCode = invoice.customerTaxCode.length > 0 ? invoice.customerTaxCode[0] :
null;
  const customerCompanyName = invoice.customerCompanyName.length > 0 ?
invoice.customerCompanyName[0] : null;
  const customerCompanyAddress = invoice.customerCompanyAddress.length > 0 ?
invoice.customerCompanyAddress[0] : null;
  // Backend-provided buyer fields (derived from vatInfo; empty when retail invoice)
  const buyerName = invoice.buyerName || "";
  const buyerAddress = invoice.buyerAddress || "";
  // Determine buyer info based on isRetailInvoice flag from backend
  let cusName;
  let cusAddress;
  let cusTaxCode;
  let cusUnitName;
  if (invoice.isRetailInvoice) {
    // Retail receipt (Bán cho người tiêu dùng) — no MST, no company info
    cusName = 'Bán cho người tiêu dùng';
    cusAddress = "";
    cusTaxCode = "";
    cusUnitName = "";
  } else {
    // VAT invoice — use buyer fields from vatInfo (forwarded as buyerName/buyerAddress)
    // and company fields from customerCompanyName/customerCompanyAddress
    cusName = buyerName || customerCompanyName || 'Bán cho người tiêu dùng';
    cusAddress = buyerAddress || customerCompanyAddress || "";
    cusTaxCode = customerTaxCode || (vatTaxCode || "");
    cusUnitName = customerCompanyName || "";
  }
  const vatRate = Number(config.vatRate) || 10;
  // Map vatRate (percent) to BKAV taxRateID per eHoadon spec:
  // 1 = 0%, 2 = 5%, 3 = 10%, 4 = 8%. Default to 3 (10%) when no match.
  const taxRateID = (() => {
    switch (vatRate) {
      case 0: return 1;
      case 5: return 2;
      case 8: return 4;
    }
  })();
}

```

```

    case 10: return 3;
    default: return 3;
  }
})();
// invoiceSerial: in production mode (useDemo=false) use config.prodInvoiceSerial
// when provided; otherwise leave " so BKAV auto-assigns via CmdType 100.
// In demo mode, leave " (BKAV demo auto-assigns).
const invoiceSerial = (!config.useDemo && config.prodInvoiceSerial)
  ? config.prodInvoiceSerial
  : "";
const invoiceDate = new Date(Number(invoice.createdAt / BigInt(1_000_000)));
// Format as ISO datetime without timezone suffix (BKAV expects this format)
const dateStr = invoiceDate.toISOString().replace('Z', "");
// Build the inner JSON payload — all keys must be camelCase as BKAV requires.
// invoiceForm is always "" — BKAV auto-assigns via CmdType 100.
// partnerInvoiceID is always 0 — only partnerInvoiceStringID carries the order reference.
const jsonPayload = {
  cmdType: 100,
  commandObject: [{
    invoice: {
      invoiceTypeID: 1,
      invoiceDate: dateStr,
      buyerName: cusName,
      buyerTaxCode: cusTaxCode,
      buyerUnitName: cusUnitName,
      buyerAddress: cusAddress,
      buyerBankAccount: "",
      payMethodID: 3,
      receiveTypeID: 1,
      receiverEmail: vatEmail || "",
      receiverMobile: "",
      receiverAddress: "",
      receiverName: "",
      note: "",
      billCode: "",
      currencyID: 'VND',
      exchangeRate: 1.0,
      invoiceStatusID: 1,
      invoiceForm: "",
      invoiceSerial: invoiceSerial,
      invoiceNo: 0,
      signedDate: '0001-01-01T00:00:00',
      typeCreateInvoice: 0,
    },
  ]},
  listInvoiceDetailsWS: invoice.items.map(item => {
    const qty = Number(item.quantity);
    const price = Number(item.price);
    const amount = qty * price;
    const taxAmount = Math.round(amount * vatRate / 100);
    return {

```

```

        itemTypeID: 0,
        itemName: item.name,
        unitName: (item.unit.length > 0 ? item.unit[0] : null) || 'Phần',
        qty: qty,
        price: price,
        amount: amount,
        taxRateID: taxRateID,
        taxAmount: taxAmount,
        isDiscount: false,
    };
    }},
    partnerInvoiceID: 0,
    partnerInvoiceStringID: String(orderId),
}
};
return jsonPayload;
}
// — BKAV API Call (via decrypt proxy)

```

```

// Sends a JSON POST to the local decrypt proxy, which forwards to the BKAV
// endpoint and decrypts the AES-256-CBC(gzip(XML)) response back to plaintext.
// The HTTP body wrapper uses { partnerGUID, partnerToken, CommandData } where
// CommandData is Base64 of the inner JSON payload. partnerToken is required by
// BKAV eHoadon JSON API CmdType 100 for authentication. Note: the outer wrapper
// key stays "CommandData" (PascalCase) per BKAV JSON endpoint spec; only the
// encoded inner payload uses camelCase fields.
// callBkav now routes through the local decrypt proxy at /opt/bkav-proxy/server.js
// (port 3000, exposes /bkav-prod and /bkav-demo). The proxy forwards the request
// to the real BKAV endpoint, decrypts the AES-256-CBC(gzip(XML)) response, and
// returns plaintext JSON to the worker. The proxy expects the same body shape
// that BKAV's JSON endpoint requires: { partnerGUID, partnerToken, CommandData }.
// `endpoint` here is the proxy URL (http://127.0.0.1:3000/bkav-prod or /bkav-demo).
function callBkav(endpoint, partnerGUID, partnerToken, jsonPayload) {
    return new Promise((resolve, reject) => {
        const jsonStr = JSON.stringify(jsonPayload);
        const base64Data = Buffer.from(jsonStr, 'utf8').toString('base64');
        // BKAV eHoadon JSON API CmdType 100 requires partnerToken in the body
        // alongside partnerGUID and CommandData for authentication.
        const httpBody = JSON.stringify({
            partnerGUID: partnerGUID,
            partnerToken: partnerToken,
            CommandData: base64Data,
        });
        const url = new URL(endpoint);
        const body = Buffer.from(httpBody, 'utf8');
        const isHttps = url.protocol === 'https:';
        const transport = isHttps ? https : http;
        const defaultPort = isHttps ? 443 : 80;
        const opts = {
            hostname: url.hostname,

```

```

    port: url.port || defaultPort,
    path: url.pathname + (url.search || ''),
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Content-Length': body.length,
    },
    rejectUnauthorized: false, // proxy/BKAV may have self-signed cert
  };
  const req = transport.request(opts, (res) => {
    const chunks = [];
    res.on('data', (c) => chunks.push(c));
    res.on('end', () => resolve(Buffer.concat(chunks).toString('utf8')));
  });
  req.setTimeout(30000, () => {
    req.destroy();
    reject(new Error('BKAV request timeout'));
  });
  req.on('error', reject);
  req.write(body);
  req.end();
});
}
// — BKAV Response Parser

```

// BKAV returns a SOAP XML envelope even for the JSON endpoint.

// ExecCommandResult contains a Base64-encoded JSON response.

```

function parseBkavResponse(xmlText) {
  log('info', 'BKAV raw response: ' + xmlText.substring(0, 300));
  // Check for SOAP fault — use [\s\S] to match across newlines (dot does not match \n)
  const faultMatch =
xmlText.match(/<(?:[^\>]+):?faultstring[^\>]*>([\s\S]*?)<\V(?:[^\>]+):?faultstring>/i);
  if (faultMatch) {
    return { success: false, errorMessage: 'SOAP fault: ' + faultMatch[1].trim() };
  }
  // Extract ExecCommandResult value from SOAP XML
  const execResultMatch =
xmlText.match(/<(?:[^\>]+):?ExecCommandResult[^\>]*>(.*?)<\V(?:[^\>]+):?ExecCommandResult>/i);
  ;
  if (!execResultMatch) {
    return { success: false, errorMessage: 'BKAV returned empty response' };
  }
  const execResult = execResultMatch[1].trim();
  // Base64 decode the result
  let jsonStr;
  try {
    jsonStr = Buffer.from(execResult, 'base64').toString('utf8');
  } catch (err) {
    return { success: false, errorMessage: 'Failed to decode BKAV response: ' + err.message };
  }
}

```



```

log('debug', 'BKAV decoded response: ' + jsonStr);
// Parse JSON
let parsed;
try {
  parsed = JSON.parse(jsonStr);
} catch (err) {
  return { success: false, errorMessage: 'Failed to parse BKAV JSON response: ' + err.message };
}
// BKAV returns Status=0 for success, non-zero for error.
// On success, parsed.Object contains the issued invoice fields:
// invoiceNo, invoiceDate (ISO/datetime), maCQT (mã cơ quan thuế), maTraCuu (mã tra cứu).
// Extract all of them so sendCallbackToCanister can forward the full set to the canister.
const success = parsed.Status === 0;
const obj = parsed.Object || {};
// Object may be a string (legacy/error) or an object (success with invoice fields).
const objFields = (obj && typeof obj === 'object') ? obj : {};
return {
  success,
  invoiceNo: success ? String(objFields.invoiceNo ?? obj ?? "") : null,
  invoiceDate: success ? (objFields.invoiceDate ? String(objFields.invoiceDate) : null) : null,
  maCQT: success ? (objFields.maCQT ? String(objFields.maCQT) : null) : null,
  maTraCuu: success ? (objFields.maTraCuu ? String(objFields.maTraCuu) : null) : null,
  error: !success ? String(parsed.Object ?? parsed.ErrorMessage ?? 'Unknown BKAV error') : null,
  errorCode: String(parsed.Status),
};
}
// — Callback to Canister

```

```

function sendCallbackToCanister(orderId, status, data) {
  return new Promise((resolve, reject) => {
    const callbackUrl = `https://${CANISTER_ID}.raw.icp0.io/invoice-callback`;
    // orderId is BigInt from Candid Nat — convert to string for JSON
    const payload = { orderId: orderId.toString(), status, ...data };
    const body = JSON.stringify(payload);
    // HMAC-SHA256 auth: canister computes HMAC-SHA256(secret, body) and compares digests
    const hmac = crypto.createHmac('sha256',
INVOICE_CALLBACK_SECRET).update(body).digest('hex');
    const url = new URL(callbackUrl);
    const opts = {
      hostname: url.hostname,
      port: url.port || 443,
      path: url.pathname,
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Content-Length': Buffer.byteLength(body),
        'X-Invoice-Signature': hmac,
      },
      rejectUnauthorized: true,
    };
  });
}

```

```

const req = https.request(opts, (res) => {
  const chunks = [];
  res.on('data', (c) => chunks.push(c));
  res.on('end', () => {
    const responseBody = Buffer.concat(chunks).toString('utf8');
    if (res.statusCode >= 200 && res.statusCode < 300) {
      resolve(responseBody);
    } else {
      reject(new Error(`Callback HTTP ${res.statusCode}: ${responseBody}`));
    }
  });
});
req.setTimeout(30000, () => {
  req.destroy();
  reject(new Error('Callback request timeout'));
});
req.on('error', reject);
req.write(body);
req.end();
});
}
// — Invoice Processing

```

```

async function processInvoice(invoice, config) {
  const orderId = invoice.orderId;
  // Route through the local decrypt proxy at /opt/bkav-proxy/server.js (port 3000).
  // The proxy exposes /bkav-prod (production) and /bkav-demo (sandbox), forwards the
  // request to the real BKAV endpoint, decrypts the AES-256-CBC(gzip(XML)) response,
  // and returns plaintext JSON to the worker. The proxy reads the real BKAV endpoint
  // from config.bkavProdEndpoint / config.bkavDemoEndpoint to know where to forward.
  const useDemo = config.useDemo;
  const PROXY_HOST = '127.0.0.1';
  const PROXY_PORT = 3000;
  const proxyPath = useDemo ? '/bkav-demo' : '/bkav-prod';
  const proxyEndpoint = `http://${PROXY_HOST}:${PROXY_PORT}${proxyPath}`;
  log('info', `Processing invoice ${orderId}`, {
    route: useDemo ? 'demo' : 'prod',
    proxy: proxyEndpoint,
    bkavEndpoint: useDemo ? config.bkavDemoEndpoint : config.bkavProdEndpoint,
  });
  const partnerGUID = config.partnerGUID || '';
  const partnerToken = config.partnerToken || '';
  let lastError = '';
  let lastErrorCode = null;
  for (let attempt = 1; attempt <= MAX_RETRIES; attempt++) {
    try {
      const jsonPayload = buildJsonPayload(invoice, config);
      const response = await callBkav(proxyEndpoint, partnerGUID, partnerToken, jsonPayload);
      const result = parseBkavResponse(response);
      if (result.success) {

```

```

log('info', `Invoice ${orderId} issued successfully`, {
  invoiceNo: result.invoiceNo,
  invoiceDate: result.invoiceDate,
  maCQT: result.maCQT,
  maTraCuu: result.maTraCuu,
});
await sendCallbackToCanister(
  orderId,
  'issued',
  {
    invoiceNo: result.invoiceNo,
    invoiceDate: result.invoiceDate,
    maCQT: result.maCQT,
    maTraCuu: result.maTraCuu,
  }
);
return;
} else {
  lastError = result.error || result.errorMessage || 'Unknown BKAV error';
  lastErrorCode = result.errorCode ?? null;
  log('warn', `Invoice ${orderId} BKAV error (attempt ${attempt}/${MAX_RETRIES})`, {
    error: lastError,
    errorCode: lastErrorCode,
  });
  if (attempt < MAX_RETRIES) {
    await new Promise((r) => setTimeout(r, RETRY_DELAY_MS));
  }
}
} catch (err) {
  lastError = err.message;
  log('error', `Invoice ${orderId} exception (attempt ${attempt}/${MAX_RETRIES})`, {
    error: lastError,
  });
  if (attempt < MAX_RETRIES) {
    await new Promise((r) => setTimeout(r, RETRY_DELAY_MS));
  }
}
}
// All retries exhausted
log('error', `Invoice ${orderId} failed after ${MAX_RETRIES} retries`, { error: lastError, errorCode:
lastErrorCode });
await sendCallbackToCanister(
  orderId,
  'error',
  { errorMessage: lastError, errorCode: lastErrorCode }
);
}
// — Poll Loop

```

```

async function pollAndProcess() {
  log('info', 'pollAndProcess: start');
  try {
    const config = await actor.getInvoiceWorkerConfig();
    const guidLen = config.partnerGUID ? config.partnerGUID.length : 0;
    const guidVal = config.partnerGUID ? `${config.partnerGUID.substring(0, 8)}...` : '';
    log('info', `Config fetched: useDemo=${config.useDemo} partnerGUID_length=${guidLen}
partnerGUID_value=${guidVal}`);
    if (!config.partnerGUID || !config.partnerToken) {
      log('warn', 'BKAV credentials not configured - skipping poll');
      return;
    }
    const pending = await actor.getPendingInvoices();
    log('info', `Pending invoices: ${pending.length}`);
    for (const invoice of pending) {
      if (isDuplicate(invoice.orderId)) {
        log('debug', 'Skipping dedup orderId: ' + invoice.orderId);
        continue;
      }
      markProcessing(invoice.orderId);
      // Process without await so poll loop is not blocked
      processInvoice(invoice, config).catch((err) =>
        log('error', 'Unhandled processInvoice error', {
          orderId: invoice.orderId,
          err: err.message,
        })
      );
    }
  } catch (err) {
    log('error', 'Poll error: ' + err.message, { stack: err.stack });
  }
}
// — Main

```

```

log('info', 'Starting BKAV invoice worker');
log('info', `Canister ID: ${CANISTER_ID}`);
log('info', `IC Host: ${IC_HOST}`);
log('info', `Poll interval: ${POLL_INTERVAL_MS}ms`);
// Initial poll with 30s timeout so setInterval always starts even if first poll hangs
const initialPollTimeout = new Promise( (_, reject) =>
  setTimeout(() => reject(new Error('Initial poll timed out after 30s')), 30000)
);
try {
  await Promise.race([pollAndProcess(), initialPollTimeout]);
} catch (err) {
  log('error', 'Initial poll failed, interval will retry: ' + err.message);
}
const intervalId = setInterval(pollAndProcess, POLL_INTERVAL_MS);
// Graceful shutdown

```

```
process.on('SIGTERM', () => {
  log('info', 'SIGTERM received, shutting down');
  clearInterval(intervalId);
  process.exit(0);
});
process.on('SIGINT', () => {
  log('info', 'SIGINT received, shutting down');
  clearInterval(intervalId);
  process.exit(0);
});
```